

LINUX

Part 1 – What is Linux?

Linux is an operating system.

An operating system is software that:

- Manages files
- Manages programs
- Manages users
- Manages hardware resources
- Allows us to interact with the computer

For data engineers, Linux is used to:

- Run data scripts
- Store data files
- Move files between directories
- Inspect logs
- Schedule jobs
- Connect to servers

Most production data systems run on Linux servers.

Part 2 – Terminal and Shell

Terminal:

The interface where you type commands.

Shell:

The program that reads your commands and executes them.

Most common shell: bash

When you type:

`ls`

The shell:

1. Reads your command
2. Finds the program named `ls`
3. Executes it
4. Displays output

Important Concept: Commands Have Structure

Most commands follow:

`command [options] [arguments]`

Example:

`ls -l /home`

- `ls` → command
- `-l` → option
- `/home` → argument

Part 3 – Understanding the Linux File System (15–20 mins)

Linux has a hierarchical structure.

Everything starts at root:

`/`

This is the top-level directory.

All other folders exist inside it.

Key Directories (Explain Slowly)

`/home`

User files.

`/etc`

Configuration files.

`/var/log`

System logs.

`/tmp`

Temporary files.

Demonstration

```
cd /  
ls
```

Then:

```
cd /home  
pwd
```

Explain:

`pwd` shows where you are.

Important Concept: Absolute vs Relative Paths

Absolute path:

Starts from /

Example:

```
/home/user/file.csv
```

Relative path:

Relative to current directory.

Example:

```
file.csv
```

Exercise (10 mins)

1. Create structure:

```
mkdir -p practice/data/raw  
cd practice  
pwd
```

2. Navigate using relative paths:

```
cd data/raw  
cd ../../
```

What does .. mean?

Answer:

Parent directory.

What does . mean?

Answer:

Current directory.

Part 4 – Listing Files

Run:

```
ls -l
```

Output looks like:

```
-rw-r--r--  1 user  user  2345 Jan 1 10:00 file.csv
```

Explain each column:

1. File type + permissions
2. Number of links
3. Owner
4. Group
5. File size
6. Last modified date
7. File name

Explain:

- → regular file
d → directory

Now run:

```
ls -lah
```

Explain:

- -a shows hidden files
- -h shows human-readable sizes

Hidden files start with .

Example:

```
ls -a
```

Part 5 – Creating and Managing Files

Create file:

```
touch file1.txt
```

Create directory:

```
mkdir logs
```

Copy file:

```
cp file1.txt backup.txt
```

Move file:

```
mv file1.txt logs/
```

Rename file:

```
mv backup.txt newname.txt
```

Delete file:

```
rm newname.txt
```

Delete directory:

```
rm -r logs
```

`rm` permanently deletes.

There is no recycle bin.

1. Using `rm -rf /` (dangerous)
2. Forgetting quotes for filenames with spaces
3. Confusing relative and absolute paths

4. Running commands in wrong directory

Mini Challenge

1. Create directory project
2. Inside create:
 - a. data
 - b. scripts
 - c. logs
3. Create 3 files inside data
4. Move one file to logs
5. Rename one file
6. Delete one file

Which command did you use to move?

Which command did you use to rename?

What happens if file does not exist?

Working With Files & Inspecting Data

Why Data Engineers Must Inspect Files

Before writing pipelines, you must be able to:

- Open files
- Inspect format
- Count rows
- Check headers
- Detect bad records
- Identify delimiter problems
- Verify file size

If you cannot inspect data manually, you cannot debug pipelines later.

Part 2 – Creating Sample Data

We will simulate a real CSV file.

Create a directory:

```
mkdir -p practice/data  
cd practice/data
```

Create sample file:

```
echo "id,name,amount" > sales.csv  
echo "1,Alice,100" >> sales.csv  
echo "2,Bob,200" >> sales.csv  
echo "3,Charlie,300" >> sales.csv
```

Open it:

```
cat sales.csv
```

Explain:

- > overwrites
- >> appends

Part 3 – Viewing Files Properly

1. cat

```
cat sales.csv
```

Use when file is small.

2. less

```
less sales.csv
```

- Arrow keys to scroll
- Press q to quit

Use for large files.

3. head

```
head sales.csv
```

Shows first 10 lines.

Specify lines:

```
head -n 2 sales.csv
```

4. tail

```
tail sales.csv
```

Shows last 10 lines.

Specify:

```
tail -n 1 sales.csv
```

5. tail -f (Real Production Use)

Simulate log:

```
touch app.log
```

In one terminal:

```
tail -f app.log
```

In another:

```
echo "New record processed" >> app.log
```

```
tail -f monitors live logs.
```

This is how you debug running jobs.

Part 4 – Counting & Measuring Data

Count lines:

```
wc -l sales.csv
```

Count words:

```
wc -w sales.csv
```

Count characters:

```
wc -c sales.csv
```

Explain:

Line count is important in pipelines to validate record counts.

Part 5 – Searching Data

1. Basic grep

```
grep Alice sales.csv
```

Case insensitive:

```
grep -i alice sales.csv
```

2. Count matching rows

```
grep Alice sales.csv | wc -l
```

3. Inverse search

```
grep -v Bob sales.csv
```

Shows rows NOT containing Bob.

Part 6 – Combining Commands

Introduce piping:

```
grep Alice sales.csv | wc -l
```

Explain:

| sends output of one command into another.

Another example:

```
cat sales.csv | head -n 2
```

Better practice:

```
head -n 2 sales.csv
```

Part 7 – Redirection

Overwrite file:

```
echo "Test" > file.txt
```

Append:

```
echo "Another line" >> file.txt
```

Redirect output:

```
ls > list.txt
```

Redirect errors:

```
ls wrongfile 2> error.txt
```

2> captures error output.

Challenge

Create a file with 50 rows.

Tasks:

1. Count rows.
2. Show first 5.
3. Show last 3.
4. Search for specific value.
5. Count occurrences.
6. Save matching rows to new file.

Permissions, Processes & Basic System Monitoring

Part 1 – Understanding Linux Permissions

1. Why Permissions Matter in Data Engineering

In real environments:

- A script may fail because it's not executable.
- A pipeline may fail because it cannot read a file.
- A job may fail because it cannot write to a directory.
- Logs may not generate because folder is not writable.

Understanding permissions prevents silent failures.

2. Reading Permission Output

Run:

```
ls -l
```

Example output:

```
-rw-r--r--  1 user  user  1234 Jan 1 10:00 sales.csv
```

First character

- - → regular file
- d → directory

Next 9 characters

Grouped into 3 sets of 3:

```
rw-  r--  r--
```

Section	Meaning
rw-	Owner permissions
r--	Group permissions
r--	Others permissions

3. Permission Types

Symbol	Meaning
r	read
w	write
x	execute

Explain carefully:

For files:

- r → read file
- w → modify file
- x → execute file (script/program)

For directories:

- r → list contents
- w → create/delete files inside
- x → enter directory

This difference is very important.

4. Modifying Permissions

Make script executable:

```
chmod +x script.sh
```

Remove write permission:

```
chmod -w sales.csv
```

Numeric method:

```
chmod 755 script.sh
```

r	4
w	2
x	1

So:

rwX = 7
r-x = 5
r-- = 4

Mini Exercise

1. Create file:

```
touch test.txt
```

2. Remove all permissions:

```
chmod 000 test.txt
```

3. Try opening it.
4. Fix it:

```
chmod 644 test.txt
```

Why did it fail earlier?

No read permission.

5. Ownership

Check owner:

```
ls -l
```

Change owner (if allowed):

```
chown user:user file.txt
```

Explain:

In production:

Services run under specific users.

If ownership is wrong → job fails.

Part 2 – Understanding Processes

1. What is a Process?

A process is a running program.

Examples:

- Bash shell
- Python script
- Sleep command
- Background job

Every running task has:

- Process ID (PID)
- CPU usage
- Memory usage

2. Viewing Running Processes

Basic view:

```
ps
```

Detailed view:

`ps aux`

- USER
- PID
- %CPU
- %MEM
- COMMAND

3. Live Monitoring

`top`

- CPU usage
- Memory usage
- Running processes
- Load average

4. Killing a Process

Start long process:

`sleep 300`

Open another terminal:

`ps aux | grep sleep`

Kill it:

`kill PID`

Force kill:

`kill -9 PID`

Use `-9` only if normal kill does not work.

Exercise

1. Start 2 sleep commands in background:

```
sleep 200 &  
sleep 300 &
```

2. Find them.
3. Kill one.
4. Verify it's gone.

Part 3 – Monitoring System Health

1. Checking CPU Usage

Open:

```
top
```

- %CPU
- Load average
- Running tasks

Load average explanation (simple):

If machine has 4 cores:

Load around 4 → fully busy

Load above 4 → overloaded

2. Checking Memory

```
free -m
```

- total
- used
- free
- available

If free memory is very low and system slow:
Likely memory pressure.

3. Checking Disk Usage

`df -h`

Shows:

- Total disk
- Used disk
- Available space

If disk hits 100%:
Programs cannot write.
Logs fail.
Pipelines fail.

4. Finding Large Files

`du -sh *`

Shows size of directories.

This is useful when disk is almost full.

Real Scenario Discussion

Scenario 1:

Script runs fine manually but fails in scheduled job.

Possible causes:

- Permission issue
- Wrong path
- Missing execute permission

Scenario 2:

Job very slow today.

Possible causes:

- CPU overloaded
- Memory low
- Disk full

What would you check first?

Shell Scripting (expanded, structured lab, real automation workflow)

What is Shell Scripting?

A shell script is a file that contains multiple Linux commands written in sequence.

Instead of typing commands one by one manually, we:

- Write them into a file
- Make the file executable
- Run it as a program

This is automation.

In data engineering, shell scripts are used to:

- Move files
- Validate data
- Trigger jobs
- Clean directories
- Schedule workflows
- Run Python scripts

Part 2 – Writing Your First Script

Create a script:

```
nano hello.sh
```

Add:

```
#!/bin/bash
```

```
echo "Hello Data Engineers"  
date
```

Save and exit.

Make it executable:

```
chmod +x hello.sh
```

Run it:

```
./hello.sh
```

Explain:

`#!/bin/bash` tells system which interpreter to use.

Without `chmod +x`, script cannot run directly.

Part 3 – Variables

Modify script:

```
#!/bin/bash
```

```
NAME="Data Engineering"
```

```
TODAY=$(date)
```

```
echo "Welcome to $NAME"
```

```
echo "Today is $TODAY"
```

- Variables store values.
- No spaces around =.
- \$ is used to access variable value.

Exercise

Create script that prints:

- Current username
- Current directory

```
whoami
```

```
pwd
```

Part 4 – Conditions

Conditions allow decision-making.

Example:

```
#!/bin/bash
```

```
FILE="sales.csv"
```

```
if [ -f "$FILE" ]  
then  
    echo "File exists"  
else  
    echo "File not found"  
fi
```

-f checks if file exists.

Other useful checks:

Condition	Meaning
-f	file exists
-d	directory exists
-s	file not empty

Example – Check If File Is Empty

```
if [ -s "$FILE" ]  
then  
    echo "File has content"  
else  
    echo "File is empty"  
fi
```

Exercise

1. Create empty file.
2. Write script that checks if file is empty.
3. Add content.
4. Run script again.

Part 5 – Loops

Loops allow repeating actions.

Example:

```
#!/bin/bash
```

```
for file in *.csv
do
    echo "Processing $file"
done
```

Create sample files:

```
touch file1.csv file2.csv file3.csv
```

Run script.

Loop goes through all matching files.

Part 6 – Real Automation Workflow

Now we simulate a basic data ingestion scenario.

Scenario

We have:

```
incoming/
processed/
```

Goal:

- Move all .csv files from incoming to processed
- Print message for each file
- Ignore non-csv files

Setup

```
mkdir incoming processed
touch incoming/a.csv incoming/b.csv incoming/test.txt
```

Script: ingestion.sh

```
#!/bin/bash

for file in incoming/*.csv
do
    if [ -f "$file" ]
    then
        mv "$file" processed/
        echo "Moved $file to processed"
    fi
done

echo "Ingestion completed"
```

Make executable:

```
chmod +x ingestion.sh
```

Run:

```
./ingestion.sh
```

Check processed folder.

Part 7 – Logging

Modify script:

```
#!/bin/bash

LOG="ingestion.log"

for file in incoming/*.csv
do
    if [ -f "$file" ]
    then
        mv "$file" processed/
        echo "$(date) - Moved $file" >> $LOG
    fi
done
```

Explain:

>> appends output to log file.

This is how production jobs record activity.

Part 8 – Debugging a Script

Introduce common mistakes:

1. Missing quotes around variables.
2. Forgetting `chmod +x`.
3. Wrong directory path.
4. Running from wrong location.
5. File pattern does not match anything.

Example bug:

```
for file in incoming/*.csv
```

If no CSV files exist,
loop still runs with literal text `incoming/*.csv`.

Fix by checking:

```
[ -e "$file" ] || continue
```

Final Practical Challenge

1. Create folder structure:
 - a. project/
 - b. project/incoming
 - c. project/processed
 - d. project/logs
2. Create 3 csv files and 1 txt file.
3. Write script that:
 - a. Moves only csv files
 - b. Logs file name with timestamp
 - c. Prints summary count of files moved

Example solution outline:

```
#!/bin/bash
```

```
COUNT=0
```

```
LOG="logs/ingestion.log"
```

```
for file in incoming/*.csv
```

```
do
```

```
  [ -e "$file" ] || continue
```

```
  mv "$file" processed/
```

```
  echo "$(date) - Moved $file" >> $LOG
```

```
  COUNT=$((COUNT+1))
```

```
done
```

```
echo "Total files moved: $COUNT"
```